

Building a Correct Sync Pipeline for HubSpot, Google Calendar and a Payments Provider

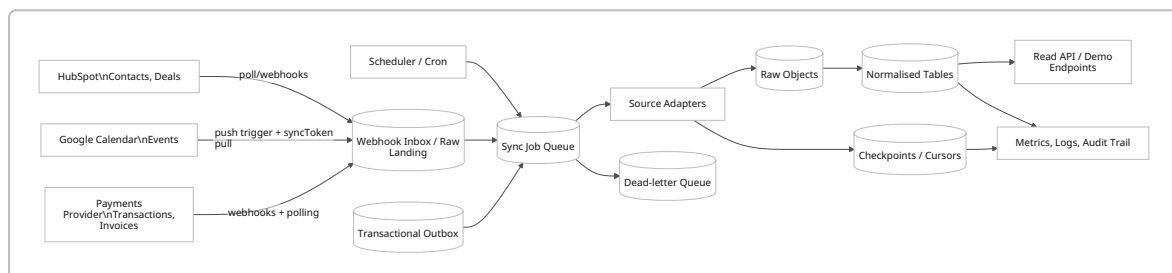
Executive summary

Problem Statement 1 is fundamentally a **data correctness** exercise, not a connector exercise. The assignment explicitly asks for a pipeline that normalises differently shaped records, survives stale incremental cursors by falling back to a full backfill, writes idempotently so duplicate delivery does not create duplicate rows, and keeps ingesting from healthy sources even when one source is down or malformed. It also requires a live deployment on Render and a demo that includes at least one failure or edge case. [filecite turn0file0](#)

The most robust design is a **source-adapter architecture** with three hard rules. First, treat every source as **at-least-once** and every webhook as **deduplicable**. Secondly, treat the database as the only durable source of internal truth: webhooks should land in an inbox table first, workers should advance checkpoints only after durable writes, and canonical tables should be updated with idempotent upserts. Thirdly, do not rely on a single incremental mechanism. Use **polling plus webhooks** where possible, because Google Calendar push notifications contain no payload and only tell you that something changed, HubSpot search is rate-limited and eventually visible rather than strongly real-time, and Stripe does not guarantee webhook ordering and only exposes Events for 30 days. [1](#)

For the three systems in scope, the recommended strategy differs by provider. For **Google Calendar**, use the documented `nextSyncToken` / `syncToken` flow as the primary incremental mechanism, with push notifications used only to trigger an incremental pull; deleted events are returned in incremental syncs, and `410 Gone` is the documented stale-token recovery path. For **HubSpot**, use timestamp-based polling over Contacts and Deals via CRM Search, with an overlap window and regular reconciliation because search results may lag, archived objects do not appear in search, and the search API is limited to five requests per second per account. For the **payments provider**, define a provider adapter contract, then implement Stripe as the concrete example: prefer webhooks plus list endpoints for primary objects such as PaymentIntents and Invoices, and never rely on Stripe Events alone because event delivery can be out of order and Events are only retrievable for 30 days. [2](#)

The most convincing final demo will therefore show four things: a successful end-to-end sync into one normalised schema, a duplicated webhook that produces no duplicate row, a deliberately stale Google `syncToken` that triggers automated recovery, and an induced failure in one source while the other two continue processing. Those proofs map directly to the assignment's intent and are stronger than a polished UI. [filecite turn0file0](#)



The diagram above reflects the core recommendation: **durable inbox, durable checkpoints, idempotent normalisation, and independent job execution per source**. That design is the simplest way to satisfy the assignment's "doesn't lie or duplicate data" requirement while remaining demonstrable on Render. [filecite turn0file0](#)

Assignment implications and design goals

The assignment gives enough latitude to simplify implementation, but not to simplify correctness. It explicitly says there is **no UI requirement**, that a CLI or a few endpoints is acceptable, and that the reviewers care about **data correctness and failure handling**. That means engineering effort should go into checkpointing, retries, reconciliation, and proof of idempotency rather than dashboards or front-end polish. [filecite turn0file0](#)

A sound set of design goals for this pipeline is:

- **No silent data loss.** Any invalid cursor, expired token, malformed payload, or partial source outage must surface in logs and metrics and trigger a repair path instead of silently advancing the checkpoint. This is directly aligned with the assignment's stale-cursor and partial-failure requirements. [filecite turn0file0](#)
- **No duplicate rows.** Duplicate webhook deliveries, rerun jobs, and overlapping incremental windows must converge to the same final state. The assignment calls this out explicitly, and Stripe's own webhook and idempotency guidance reinforces why an at-least-once model is the right assumption. [filecite turn0file0](#) ³
- **Per-source isolation.** One broken adapter must not wedge the entire run. The job model should therefore be source-scoped and connection-scoped, not "one giant sync transaction". That is the simplest way to satisfy the requirement that the other two sources still land even if one returns garbage. [filecite turn0file0](#)
- **Auditable recovery.** Recovery should be visible and replayable, especially for Google Calendar stale-token recovery and payments webhook duplication. Google explicitly documents [410 Gone](#) invalidation for `syncToken`, and Stripe explicitly documents retries and non-guaranteed event ordering. ⁴

A practical corollary is that the pipeline should not try to promise **exactly-once** end-to-end delivery. Across external APIs and webhooks, that is not realistic. What is realistic is **at-least-once ingestion + idempotent processing + explicit reconciliation**, which is sufficient to make the stored state correct. This is the right trade-off for the assignment.

Canonical data model and source entities

A reliable sync pipeline needs two schemas, not one. The first is a **raw ingestion schema** that preserves source payloads and delivery metadata for audit and replay. The second is a **normalised operational schema** that exposes unified records to downstream logic. If you skip the raw layer, stale-cursor recovery and demo replay become much harder.

Recommended normalised schema

The minimum durable schema should include these tables:

Table	Purpose	Recommended unique key
<code>integration_connection</code>	Per installed source account/ calendar/payment account, auth metadata, scopes, status	<code>provider + external_account_id</code>
<code>source_checkpoint</code>	One row per connection and stream, storing cursor type/ value, last successful sync time, recovery status	<code>connection_id + stream_name</code>
<code>webhook_inbox</code>	Immutable receipt log of webhook deliveries before processing	<code>provider + delivery_id</code>
<code>raw_object</code>	Latest raw payload per source object plus history pointer	<code>provider + connection_id + object_type + external_id</code>
<code>raw_object_version</code>	Optional append-only version history for replay/demo evidence	surrogate key
<code>party</code>	Canonical person/company abstraction used to link CRM, calendar, and payments	surrogate key plus resolver constraints
<code>opportunity</code>	Normalised deal / pipeline record	<code>provider + connection_id + external_id</code>
<code>calendar_event</code>	Normalised event record	<code>provider + connection_id + external_id</code>
<code>financial_transaction</code>	Normalised payment transaction / intent / charge / refund projection	<code>provider + connection_id + external_id</code>
<code>invoice</code>	Normalised invoice projection	<code>provider + connection_id + external_id</code>
<code>entity_link</code>	Cross-system linkage, for example HubSpot contact ↔ payment customer	composite
<code>sync_run</code>	Run metadata, timings, counters, errors	surrogate key
<code>dead_letter_item</code>	Poison messages and exhausted retries	surrogate key
<code>outbox</code>	Transactionally emitted internal work items	surrogate key

The key pattern is simple: every provider object should have a **stable external identity**, and every write path should upsert on that identity. Separately, store **source version markers** such as `updatedAt`,

`lastmodifieddate`, `etag`, payload hash, or event id so that stale or repeated writes can be ignored deterministically.

HubSpot entities and fields to sync

HubSpot's CRM Search documentation is unusually helpful for choosing a minimum viable field set. For Contacts, default returned properties include `firstname`, `lastname`, `email`, `lastmodifieddate`, `hs_object_id`, and `createdate`. For Deals, default returned properties include `dealname`, `amount`, `closedate`, `pipeline`, `dealstage`, `createdate`, `hs_lastmodifieddate`, and `hs_object_id`. Search responses also include `id`, `createdAt`, `updatedAt`, and `archived`. ⁵

That leads to these concrete recommendations:

HubSpot stream	Sync as required minimum	Useful additions	Why it matters
Contacts	<code>external_id</code> , <code>email</code> , <code>firstname</code> , <code>lastname</code> , <code>createdate</code> , <code>lastmodifieddate/updatedAt</code> , <code>archived</code>	phone, owner, lifecycle stage, associated company ids, associated deal ids	enough to prove identity, updates, linkage, and dedupe
Deals	<code>external_id</code> , <code>dealname</code> , <code>amount</code> , <code>closedate</code> , <code>pipeline</code> , <code>dealstage</code> , <code>createdate</code> , <code>hs_lastmodifieddate/updatedAt</code> , <code>archived</code>	owner, associated contact ids, currency, forecast category	enough to prove state transitions and normalisation
Associations	contact↔deal edges	company↔contact, company↔deal	enables cross-entity integrity

Two important trade-offs follow from the docs. First, **search is good for incremental fetches, but not sufficient alone for deletion semantics**, because archived CRM objects do not appear in search results. Secondly, **search is not strongly real-time**, because the docs warn that newly created or updated objects may take a few moments to appear. That means the safe design is a timestamp overlap window plus periodic reconciliation rather than a brittle “advance watermark exactly once” model. ⁶

Google Calendar entities and fields to sync

The Calendar Events resource includes a rich event model with fields such as `id`, `status`, `created`, `updated`, `summary`, `description`, `location`, `start`, `end`, `recurringEventId`, `originalStartTime`, `iCalUID`, `sequence`, `attendees`, `organizer`, `extendedProperties`, and `eventType`. ⁷

For this assignment, do not over-sync. A sensible normalised event record is:

Google Calendar stream	Sync as required minimum	Useful additions	Why it matters
Events	external_id, calendar_id, status, summary, start, end, updated, iCalUID, recurringEventId, originalStartTime, organizer_email	attendees, location, description, sequence, extendedProperties	enough to prove create/update/delete, recurrence, and dedupe

Two specific recommendations matter here. Store both `id` and `iCalUID`: the former is the API object id, while the latter is useful for interoperability and recurrence handling. Also store the provider's `status` and a local `deleted_at` / `source_deleted` flag so that tombstones from incremental sync can be represented without losing audit history. Google documents that incremental sync results always contain deleted entries. ⁸

Payments entities and fields to sync

Because the payments provider is unspecified, the schema must be **provider-agnostic**. Still, the assignment explicitly asks for transactions and invoices, so your adapter contract should require those two streams even if one provider models them differently. Stripe is a good concrete example:

`PaymentIntent` list responses expose `id`, `amount`, `currency`, `created`, `customer`, `status`, and related payment fields, while `Invoice` list responses expose `id`, `customer`, `customer_email`, `amount_due`, `amount_paid`, `currency`, `created`, `status`, and `status_transitions`. ⁹

Recommended canonical structures:

Payments stream	Sync as required minimum	Useful additions	Provider-specific notes needed
Transactions	external_id, kind, amount, currency, status, created_at, customer_external_id, invoice_external_id, raw_reference	captured amount, refunded amount, payment method, settlement time	whether "transaction" maps to payment intent, charge, capture, or ledger entry
Invoices	external_id, status, amount_due, amount_paid, currency, created_at, paid_at, customer_external_id	hosted URL, due date, collection method, line items	whether invoice mutations have webhooks and updated filters
Refunds / adjustments	separate table or subtype of transaction	reason, amount, created_at	required if the provider models refunds separately

The most important modelling decision is this: **do not collapse payments status vocabularies too early in Problem Statement 1**. Persist the original provider status and type, then normalise into internal enums separately. That preserves auditability and makes Problem Statement 2 easier later.

Authentication, scopes and rate limits

Authentication and rate limiting should be designed as **per-connection concerns**, not as app-wide constants. Each connection needs its own credentials, scopes, quota budget, error counters, and backoff state.

HubSpot

HubSpot recommends OAuth for secure authentication, and its OAuth flow issues both access and refresh tokens. Access tokens have an `expires_in` lifetime and must be refreshed using the refresh token rather than assuming any `401` means “refresh now”. HubSpot also notes that apps installed into a single account can use a static auth token, but OAuth is the safer default if you want an architecture that scales beyond a single developer account. ¹⁰

For scopes, HubSpot’s authentication docs explain that scopes are granular and endpoint-specific, and that required scopes are listed for each API guide. The OAuth guide gives a concrete app configuration example using `crm.objects.contacts.read` and `crm.objects.contacts.write`. For this assignment, the correct principle is **least privilege**: request only the read scopes necessary for Contacts and Deals unless you actually write back. ¹¹

HubSpot rate limits are split between general limits and API-specific limits. The general docs say marketplace OAuth apps are limited to 110 requests every 10 seconds per installed account, while privately distributed apps have limits based on subscription tier. Separately, the CRM Search API is explicitly rate-limited to **five requests per second per account**, with pages of up to **200 objects** and a hard limit of **10,000 total results per query**. ¹²

Recommendation. For HubSpot, prefer a single shared throttler per account and per stream, and partition large backfills by date windows to stay below the 10,000-result cap. The key trade-off is lower coding simplicity versus much lower risk of silent partial backfills.

Implementation checklist.

- Store `access_token`, `refresh_token`, `expires_at`, granted scopes, and HubSpot account id.
- Refresh proactively when `expires_at` is near, not reactively after random failures.
- Partition full backfills by `createdate` windows.
- Cap concurrent search requests to respect five requests per second per account.
- Add a fixed overlap window for incremental polling because search visibility can lag. ¹³

Google Calendar

Google Calendar uses OAuth scopes that are clearly documented. For read-only event syncing, `calendar.events.readonly` is sufficient for events, while `calendar.readonly` is broader and allows reading any accessible calendar metadata as well. If you need to enumerate subscriptions, `calendar.calendarlist.readonly` may also be required. Google also notes that public applications using sensitive scopes may require app verification. ¹⁴

Quota is enforced per project and per user. As of Google's current Calendar API quota page, the documented limits are **10,000 requests per minute per project** and **600 requests per minute per user per project**, calculated on a sliding window. Google's error guidance says to respond to `userRateLimitExceeded` and `rateLimitExceeded` with exponential backoff. ¹⁵

Recommendation. Use user OAuth for a personal developer calendar; use a service account with domain-wide delegation only when you are syncing a Google Workspace domain you control. Use a per-user budgeter so one noisy calendar does not starve the rest. ¹⁶

Implementation checklist.

- Store refresh tokens and scope grants per calendar connection.
- Request `calendar.events.readonly` unless broader access is genuinely needed.
- Maintain per-project and per-user token buckets.
- Back off on both `403 rateLimitExceeded` and `429` style responses.
- Track push channel expiry for renewal. ¹⁷

Payments provider and Stripe example

Because the provider is unspecified, the adapter should abstract authentication as one of three modes: **API key**, **OAuth account connection**, or **signed webhook-only plus pull credentials**. Stripe is the clean example of the API-key model: the API authenticates with secret or restricted API keys over HTTPS, and Stripe recommends restricted API keys over unrestricted secret keys where possible. Webhook signing secrets are separate from API keys. ¹⁸

Stripe's rate-limit guidance is also concrete: the global API limit is **100 requests per second in live mode** and **25 requests per second in sandbox**, with many individual endpoints limited to **25 requests per second**. `429 Too Many Requests` responses should be handled with exponential backoff and preferably a token-bucket limiter. ¹⁹

Recommendation. Design the payments adapter around a declared capability map:

- auth mode
- list streams supported
- incremental cursor type
- webhook signature scheme
- maximum page size
- deletion/tombstone behaviour
- rate-limit headers and retry-after semantics

That keeps Stripe-specific behaviour in one adapter while leaving room for PayPal, Adyen, Square, or others later.

Implementation checklist.

- Support provider-specific secret storage and rotation.
- Verify every incoming webhook signature before enqueueing work.
- Rate-limit at both provider-global and connection-local levels.
- Model auth refresh if the provider uses OAuth rather than keys.
- Keep a per-provider capability manifest in code. ²⁰

Sync design and cursor recovery

This section is the heart of the assignment. The safest mental model is:

webhooks tell you that something probably changed; polling tells you exactly what changed; full reconciliation proves what the final truth is.

Incremental sync design

Google Calendar has the cleanest documented incremental model. The official sync guide describes a one-time full sync that ends with `nextSyncToken`, followed by repeated incremental calls using `syncToken`. Google documents that incremental results always include deleted entries, that the exact same query parameters must be reused during incremental pagination, and that a `410 Gone` means the token is invalid and a fresh full sync is required. ²¹

HubSpot does not expose an equivalent opaque sync token in the CRM Search docs cited here. Instead, its documented search model gives you filterable object search, one sort rule, integer `after` paging, up to 200 objects per page, and default modified timestamps for Contacts and Deals. That makes the practical incremental pattern a **timestamp watermark with overlap**, not a pure opaque cursor. Because HubSpot also warns that newly created or updated objects may take a few moments to appear, the overlap window is a correctness feature, not an optimisation. ⁵

For the payments provider, your design should branch by capability. With Stripe, object list endpoints support `created` filters and cursor pagination via `starting_after` / `ending_before`, while Events can be listed for up to 30 days. That suggests a hybrid strategy: use webhooks and, where helpful, Events for short-term detection, but always rehydrate the canonical object using the primary object endpoint and never rely on Events as your only durable incremental source. ²²

Full sync strategy

A full sync should not mean “one giant unbounded fetch”. It should mean **bounded partitions with resumable checkpoints**.

For HubSpot, the 10,000-result cap on any single search query means your initial Contacts and Deals backfill should be partitioned, typically by `createdate` windows such as month or week. For Google Calendar, the initial sync may be date-bounded if your demo only needs recent events, but you must store the `nextSyncToken` from the final page. For Stripe-like payments providers, use `created` windows and page cursors, because list endpoints are sorted and paginated. ²³

Conflict resolution recommendation. Treat provider-sourced fields as externally authoritative. If you need internal annotations, store them in separate extension columns or tables. Do not overwrite provider state with local edits in the same columns, or stale recovery becomes ambiguous. This is especially important when a backfill rehydrates older rows after a cursor repair.

Webhooks versus polling

The table below is a synthesis of the source documentation and the practical design patterns that follow from it.

Approach	Advantages	Weaknesses	Best use in this assignment
Webhooks	Lowest latency; lower API cost; good trigger mechanism	Duplicate delivery is normal; providers may retry; ordering may not be guaranteed; payloads may be partial or empty	Use for Stripe and Google trigger paths, and optionally HubSpot if available
Polling	Deterministic, replayable, easier to reason about checkpointing	Higher API load; slower freshness; can hit rate limits	Use as the primary truth path for HubSpot and as the repair path for all providers
Hybrid	Best correctness/freshness balance	More moving parts	Recommended overall design

This recommendation is grounded in provider behaviour: Google push notifications contain no message body and only tell you to fetch change details separately; Stripe retries webhook deliveries and does not guarantee event ordering; HubSpot's general API guidance itself recommends using webhooks to reduce API traffic, while the search docs show the limits of polling. ²⁴

Comparing cursor types

Cursor type	Example provider use	Strengths	Weaknesses	Recommendation
Opaque sync token	Google Calendar <code>nextSyncToken</code> / <code>syncToken</code>	Server-managed change tracking; includes deletions	Can expire or be invalidated; requires strict query consistency	Use when offered; keep recovery path for <code>410</code>
Timestamp watermark	HubSpot <code>lastmodifieddate</code> / <code>hs_lastmodifieddate</code> pattern	Easy to understand and partition	Vulnerable to equal timestamps, visibility lag, and clock assumptions	Use with overlap windows and idempotent upserts
Page cursor	HubSpot <code>after</code> , Stripe <code>starting_after</code>	Good for bounded page traversal	Not a long-term incremental checkpoint by itself	Use only inside a single run
Event-log cursor	Stripe Events id or created window	Useful for short-term replay and webhook repair	Event retention may be limited; ordering may not be guaranteed	Use as a helper, not sole source of truth

Google's sync guide documents opaque token invalidation and recovery. HubSpot's search guide documents `after` paging and modified timestamps. Stripe documents both page cursors and the fact that Events are only accessible for 30 days. ²⁵

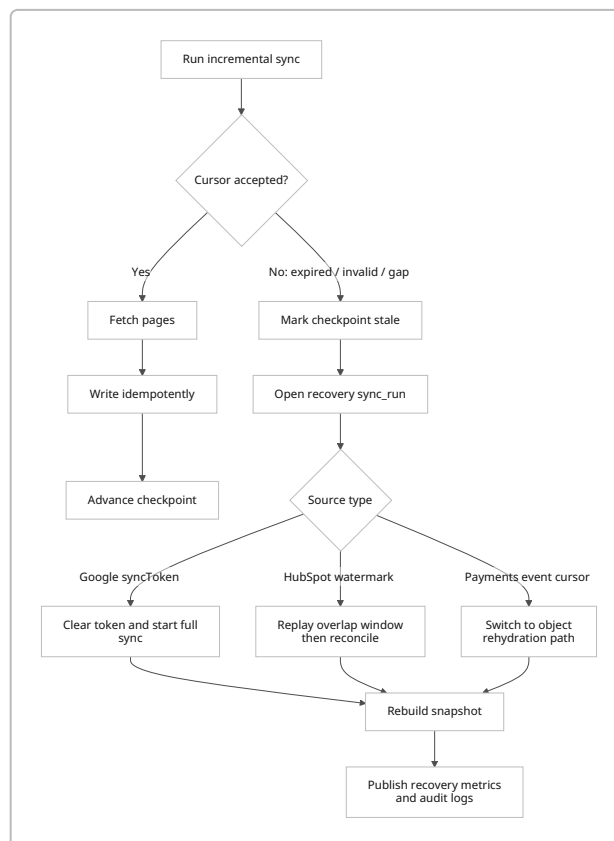
Stale cursor detection and recovery

The recovery path should be source-specific.

Google Calendar. This is the most explicit case. If an incremental request returns `410 Gone`, delete the saved sync token, logically invalidate the local snapshot, and start a new full sync. Google explicitly describes this as the required behaviour. Also, renew push notification channels before expiry because Google does not renew them automatically. ²⁶

HubSpot. Because the practical cursor is a timestamp watermark rather than a provider-issued opaque token, stale-cursor detection is heuristic. Recommended signals are: the checkpoint is moving but record counts collapse unexpectedly; webhook counts exceed polled deltas over a window; or a periodic reconciliation finds missing or archived objects. Because search results may lag and archived objects do not appear in search, the repair path should be an **overlap replay first**, then a **windowed full reconcile**, not necessarily a destructive wipe. ⁶

Payments provider. If the provider only offers event-log backfills and event retention is finite, stale detection should compare the saved cursor time to the provider's event-retention horizon. For Stripe, Events are retrievable for 30 days only, so a worker resuming after that window should mark the event cursor unusable and rehydrate from object list endpoints for the affected date windows. ²⁷



This flow is what you should actually replay in the demo: it is visible, understandable, and maps directly to the assignment wording. [filecite turn0file0](#)

Idempotency, failure handling, consistency and observability

Idempotent writes

There are three separate idempotency problems, and each needs its own key.

Layer	Key	Pattern	Why
Webhook receipt	provider delivery id, or signed payload hash if absent	insert into <code>webhook_inbox</code> with unique constraint	prevents duplicate deliveries from double-enqueueing
Raw object storage	<code>provider + connection + object_type + external_id</code>	upsert current version; optionally append history	ensures one latest row per provider object
Canonical projection	same external identity plus version/timestamp guard	upsert only if new version is newer or payload hash differs	prevents older/out-of-order deliveries from regressing state
Outbound API mutations	provider idempotency key	send on writes to providers	protects retries of provider mutations

Stripe's idempotency documentation makes the outbound part explicit: all `POST` requests accept idempotency keys, Stripe stores the first result for a key, recommends UUIDv4-style keys, and may prune keys after at least 24 hours. [28](#)

Recommendation. Implement all internal writes as database upserts guarded by a source version field, for example `source_updated_at`, `etag`, `sequence`, or payload hash. The biggest trade-off is a bit more schema complexity upfront, but it removes whole classes of duplicate-row bugs later.

Implementation checklist.

- Unique index on `webhook_inbox(provider, delivery_id)`.
- Unique index on every canonical table's external identity tuple.
- Upsert guard: only update when incoming version is newer or different.
- Retain raw payload hash for duplicate-proof demo evidence.
- Use provider idempotency keys on any outbound mutation path. [29](#)

Partial failures, retries and dead letters

The assignment explicitly says the pipeline must keep going when one source is down or returns garbage. The simplest architecture that satisfies this is **one job per source connection per stream**, each with its own retries and checkpoint. [filecite turn0file0](#)

For retries, use **exponential backoff with jitter** for provider rate limits and transient failures. Google explicitly recommends exponential backoff for rate-limit errors. Stripe does the same for rate limiting, and Stripe's webhook delivery system itself uses exponential backoff for automatic retries. ³⁰

The recommended database pattern is a **transactional outbox**:

1. insert webhook into `webhook_inbox`;
2. in the same transaction, insert an outbox row;
3. a worker reads outbox rows and performs the heavy sync work;
4. if processing exceeds retry policy, move the item to the dead-letter queue.

This pattern is particularly important for webhook handlers because both Stripe and Google recommend returning a successful response quickly rather than doing heavy work inline. Stripe says to return `2xx` before complex logic, and Google requires a successful status for notification handling while the full change details must be fetched separately anyway. ³¹

Consistency and ordering guarantees

You should promise **eventual consistency with replay-safe convergence**, not serialisable global ordering.

Why:

- Google push message numbers increase but are not sequential, and messages contain no object payload, so they are not a reliable object-state log. ³²
- Stripe explicitly does **not** guarantee webhook event ordering. ³³
- HubSpot search visibility may lag briefly. ⁶

That yields these concrete guarantees:

- **Per-record last-write-wins using provider version markers.**
- **Per-source checkpoints advance only after durable writes succeed.**
- **No single global ordering across providers.**
- **Periodic reconciliation closes gaps that ordering cannot prevent.**

This is the right balance between correctness and implementation cost.

Monitoring and observability

The observability model should be designed around proving correctness, not merely uptime.

Metrics to expose

- `sync_runs_total{provider,stream,status}`
- `sync_duration_seconds`
- `records_fetched_total`
- `records_upserted_total`
- `records_deduplicated_total`
- `records_tombstoned_total`
- `checkpoint_age_seconds`
- `webhook_lag_seconds`

- `stale_cursor_recoveries_total`
- `api_rate_limit_errors_total`
- `dead_letter_items_total`
- `source_freshness_seconds{provider}`

Logs to emit

- `run_id`, `provider`, `connection_id`, `stream`, `cursor_value`, `page_cursor`
- provider request ids and correlation ids
- diff summary: fetched/written/ignored/deleted
- recovery transitions: stale detected, full rehydrate started, full rehydrate completed
- webhook dedupe hits

Tracing Instrument webhook receipt, queue dispatch, provider API calls, and database upserts under a single trace id. Even if you do not use a full tracing backend in the demo, propagate a correlation id through logs so you can replay one stale-cursor incident from handler to repair completion.

Render provides in-dashboard metrics and health checks, and it can stream service logs to external logging providers over syslog or HTTPS. That is enough for a small demo deployment; for richer tracing, instrument the app and ship logs to an external observability service. ³⁴

Render deployment and non-functional requirements

Recommended Render process model

Render's deployment primitives map well to this pipeline:

- a **web service** for health endpoints, on-demand sync endpoints, and webhook receivers;
- a **background worker** for queue processing and provider sync jobs;
- a **cron job** for scheduled incremental syncs, stale-cursor sweeps, and reconciliation runs. ³⁵

Render describes background workers as continuously running services that do not receive incoming network traffic and typically poll a task queue. Render Cron Jobs run on schedules you define. Health checks apply to web services and private services, not to background workers. ³⁶

Recommendation. For a free-tier-friendly assignment build, use:

- one web service,
- one worker,
- one cron job,
- one Postgres database used for both application state and a lightweight queue.

That avoids introducing Redis unless you already have it. A Postgres-backed job queue plus advisory locking is enough at assignment scale.

Secrets management and configuration

Render automatically injects environment variables and exposes useful runtime values such as `RENDER_EXTERNAL_URL`, `RENDER_SERVICE_NAME`, `RENDER_INSTANCE_ID`, and `PORT`. Render's environment variable docs also confirm that values are strings and that the default web port is `10000`. ³⁷

For secrets, use environment variables supplied by the platform. Stripe's own key guidance also recommends a secrets vault where available, or environment variables if not. Never place API keys or webhook secrets in source control. ³⁸

Implementation checklist.

- Store provider secrets as Render environment variables.
- Separate test/sandbox from live credentials.
- Keep webhook signing secrets distinct from API credentials.
- Use `RENDER_EXTERNAL_URL` to construct callback URLs where appropriate.
- Expose a readiness endpoint on the web service for health checks. ³⁹

Scaling and health

Render supports scaling multiple instances of web services, private services, and background workers, with manual scaling available broadly and autoscaling available only on higher plans. Render also notes that services with attached persistent disks cannot scale to multiple instances, and health checks apply only to services that receive incoming traffic. ⁴⁰

That implies two design recommendations:

- Keep all durable state in Postgres, not on local disk, so the service can scale later.
- Add a **worker heartbeat** table and a cron-driven stuck-job detector, because the worker itself will not benefit from Render HTTP health checks. ⁴⁰

Non-functional requirements to account for

These are the non-functional requirements that matter most for this assignment implementation:

Latency.

Webhook handlers should acknowledge quickly and defer heavy work. Stripe explicitly recommends returning `2xx` before complex logic, and the same principle is appropriate for Google push notifications. A good demo target is sub-second ack and low-minutes end-to-end convergence under polling fallback. ³¹

Throughput.

Size provider concurrency to the documented limits, not to CPU. HubSpot search is five requests per second per account; Google Calendar has per-project and per-user minute quotas; Stripe publishes both global and endpoint request-rate limits. ⁴¹

Data retention.

Keep:

- raw payloads for 30–90 days,
- webhook inbox and sync runs for at least the duration needed for demo replay,
- canonical objects indefinitely or per product need,
- deleted/tombstoned records as soft deletes, not hard deletes.

This is particularly important because Stripe Events themselves are only retrievable for 30 days. ⁴²

Security.

Use least-privilege scopes, store secrets only in backend config, validate webhook signatures, and redact sensitive fields from logs. Stripe explicitly documents signature verification and key-protection guidance. Google requires HTTPS webhook endpoints for push notifications. ⁴³

Compliance.

Even in a demo, minimise PCI scope by never handling card PAN data directly; store only provider object ids and approved metadata. Also consider UK GDPR / GDPR implications if you copy contact emails and notes into your own database. Compliance posture here is mostly about scope minimisation and auditability.

Testing, demo evidence and timeline

What to test

Because the assignment is about correctness under failure, your test plan should be heavier on **state-transition tests** than on controller tests.

End-to-end tests

- Seed HubSpot Contacts and Deals, Google Calendar Events, and payments test objects.
- Run initial backfill.
- Mutate one record in each source.
- Re-run incrementals.
- Assert that canonical tables reflect updates once and only once.

Failure-injection tests

- Corrupt a saved Google `syncToken` and verify that the next sync logs a stale-token event and performs a full rehydrate.
- Replay the same Stripe webhook twice and prove only one canonical row exists.
- Return a malformed payload into the webhook receiver and prove it is quarantined without crashing the worker.
- Simulate HubSpot 429 handling and prove the other source jobs continue.
- Simulate Stripe event reordering and prove later stale events do not overwrite newer state. Google and Stripe documentation both justify these tests because Google documents token invalidation and Stripe documents retries and out-of-order delivery. ⁴

Best evidence to show in the final demo

The final demo should show evidence, not just claims. The strongest sequence is:

Evidence	What to show	Why it is convincing
Live trigger	a real Render endpoint or cron-triggered run	matches the assignment's live deployment requirement
Logs	one run id spanning fetch, write, checkpoint advance	proves durability and traceability

Evidence	What to show	Why it is convincing
Dashboard counters	fetches, writes, dedupes, stale recoveries, DLQ size	proves correctness is measurable
Duplicate webhook replay	same delivery resent twice, no extra row created	proves idempotency
Stale cursor recovery	Google 410 path and automatic full rehydrate	proves no silent data loss
Source isolation	one provider intentionally failed while others succeed	proves the run does not wedge
Raw vs canonical view	raw payload saved once, canonical upserted once	proves audit trail and convergence

Those demo elements align directly with what the assignment asks you to submit: a live deployment, a short demo video, and sources/references explaining how you researched the build.

fileciteturn0file0

Suggested five-day implementation and demo plan

Day	Focus	Deliverable
Day one	Project skeleton, Postgres schema, connection/auth storage, Render web service	health endpoint, schema migrations, seeded test accounts
Day two	HubSpot and Google adapters, initial full sync paths, checkpoint tables	Contacts, Deals, and Events landing in raw + canonical tables
Day three	Payments adapter, webhook inbox, idempotent upserts, transactional outbox	duplicate-safe Stripe-style flow and worker processing
Day four	Stale-cursor recovery, retries, DLQ, structured logs, counters	demonstrable Google 410 recovery and provider isolation
Day five	Render worker/cron deployment, script the demo, polish README and evidence capture	final five-minute demo script with logs and dashboards

A good five-minute demo script

Start by showing the live Render deployment and a `/health` or `/sync/run` endpoint. Then show seeded sample records in HubSpot, Google Calendar, and the payments sandbox. Run a sync and show the normalised tables. Next, replay a duplicate webhook and prove row counts do not increase. After that, corrupt the Google checkpoint, rerun sync, and show the recovery log path plus rebuilt event state. Finally, force one source to fail and show that the other sources still complete. That script demonstrates exactly the behaviours the assignment explicitly values. fileciteturn0file0

Final recommendations in one place

The concise design recommendation is:

- Use **Postgres as the durable state store** for connections, checkpoints, webhook inbox, raw payloads, canonical objects, outbox, and DLQ.
- Use **webhooks as triggers**, not as sole truth.
- Use **Google syncToken** exactly as documented, including **410** recovery.
- Use **HubSpot timestamp overlap polling** plus reconcile windows because CRM Search is capped, paginated, not immediate, and excludes archived results.
- Use a **payments adapter capability model**, with Stripe implemented via object polling plus signed webhooks, not Events-only ingestion.
- Acknowledge all webhook deliveries quickly, process asynchronously, and make every write idempotent.
- Demo proof should emphasise **recovery, dedupe, and isolation**, because that is what Problem Statement 1 is really testing. ⁴⁴

¹ ²⁴ ³² [Push notifications | Google Calendar | Google for Developers](https://developers.google.com/workspace/calendar/api/guides/push)

<https://developers.google.com/workspace/calendar/api/guides/push>

² ⁴ ²¹ ²⁵ ²⁶ ⁴⁴ [Synchronize resources efficiently | Google Calendar | Google for Developers](https://developers.google.com/workspace/calendar/api/guides/sync)

<https://developers.google.com/workspace/calendar/api/guides/sync>

³ ³¹ ³³ ⁴³ docs.stripe.com

<https://docs.stripe.com/webhooks>

⁵ ⁶ ¹³ ²³ ⁴¹ [Search the CRM - HubSpot docs](https://developers.hubspot.com/docs/api-reference/latest/crm/search-the-crm)

<https://developers.hubspot.com/docs/api-reference/latest/crm/search-the-crm>

⁷ ⁸ [Events | Google Calendar | Google for Developers](https://developers.google.com/workspace/calendar/api/v3/reference/events)

<https://developers.google.com/workspace/calendar/api/v3/reference/events>

⁹ docs.stripe.com

https://docs.stripe.com/api/payment_intents/list

¹⁰ ¹² [API usage guidelines and limits - HubSpot docs](https://developers.hubspot.com/docs/api/usage-details)

<https://developers.hubspot.com/docs/api/usage-details>

¹¹ [Working with OAuth - HubSpot docs](https://developers.hubspot.com/docs/apps/developer-platform/build-apps/authentication/oauth/working-with-oauth)

<https://developers.hubspot.com/docs/apps/developer-platform/build-apps/authentication/oauth/working-with-oauth>

¹⁴ [Choose Google Calendar API scopes | Google for Developers](https://developers.google.com/workspace/calendar/api/auth)

<https://developers.google.com/workspace/calendar/api/auth>

¹⁵ ¹⁷ [Usage limits | Google Calendar | Google for Developers](https://developers.google.com/workspace/calendar/api/guides/quota)

<https://developers.google.com/workspace/calendar/api/guides/quota>

¹⁶ ³⁰ [Handle API errors | Google Calendar | Google for Developers](https://developers.google.com/workspace/calendar/api/guides/errors)

<https://developers.google.com/workspace/calendar/api/guides/errors>

¹⁸ docs.stripe.com

<https://docs.stripe.com/api/authentication>

¹⁹ docs.stripe.com

<https://docs.stripe.com/rate-limits>

- 20 38 **docs.stripe.com**
<https://docs.stripe.com/keys>
- 22 **docs.stripe.com**
<https://docs.stripe.com/api/events/list>
- 27 42 **docs.stripe.com**
<https://docs.stripe.com/api/events>
- 28 29 **docs.stripe.com**
https://docs.stripe.com/api/idempotent_requests
- 34 **Health Checks – Render Docs**
<https://render.com/docs/health-checks>
- 35 36 **Background Workers – Render Docs**
<https://render.com/docs/background-workers>
- 37 39 **Default Environment Variables – Render Docs**
<https://render.com/docs/environment-variables>
- 40 **Scaling Render Services – Render Docs**
<https://render.com/docs/scaling>